



Programação Orientada a Objetos

Lista de Exercícios 01

Aula 02 – Introdução aos Aplicativos Java

- 1) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
 - a. Todas as variáveis devem ser definidas com um tipo quando declaradas
 - b. JAVA considera as variáveis „numero. e „NuMeRos. iguais.
 - c. TODAS as seguintes declarações de variáveis são válidas:
 - i. `int numero;`
 - ii. `float procura_elemento;`
 - iii. `String Representa&Exibe;`
 - iv. `String 3escolhas`
 - v. `String _3escolhas;`
- 2) Escreva um aplicativo que calcule os 100 primeiros termos da tabuada de um número n qualquer.
- 3) Escreva um aplicativo que calcule os números múltiplos de 3 entre 1 e 100. Um número é múltiplo de 3 se o resto de sua divisão por 3 é zero.
- 4) Escreva um aplicativo que leia a altura e o peso de uma pessoa, calcule seu IMC e retorne a interpretação deste resultado. Veja o cálculo de IMC em <https://como-emagrecer.com/calculo-de-imc.html>. (Por exemplo, “Seu IMC é de 28 e sua situação é que você está acima do peso”).
- 5) Escreva um aplicativo que leia um número inteiro e escreva cada um dos numerais que formam este número em uma linha diferente. Por exemplo, para o número 153 deve ser mostrado:

1
5
3

Aula 03 – Objetos e Classes

- 6) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
- Por convenção, os nomes de métodos começam com uma letra maiúscula e todas as seguintes palavras com letras maiúsculas.
 - Quando uma classe em um pacote usa outra classe no mesmo pacote, não é necessário fazer uma operação de import.
 - Uma declaração de um método seguido por parênteses vazio indica que o método não precisa de nenhuma parâmetro para realizar sua tarefa.
 - Variáveis com modificadores de acesso private são diretamente acessíveis somente pelos métodos da classe nos quais elas foram declaradas
 - Uma variável primitiva pode invocar um método.
 - Variáveis declaradas no corpo de um método são conhecidas como variáveis de instância e podem ser utilizadas por todos os métodos na classe
 - Em variáveis de instância, tanto variáveis do tipo primitivo quando não primitivos são inicializadas por seus valores padrão.
- 7) Escreva um aplicativo Java que leia um número e verifique se esse número é primo.
- 8) Escreva um aplicativo Java que leia dois números e escreva apenas os números da sequência de Fibonnacci que estão entre os dois números lidos. Considere que não necessariamente os números serão informados em ordem crescente.
- 9) Escreva um aplicativo Java que leia uma frase e imprima cada caractere seguido pelo seu código numérico. Por exemplo, considerando-se a frase “ok” o programa deve imprimir o seguinte resultado:
- o = 111
k = 107
- 10) Implemente uma classe chamada Empregado que inclua três atributos: primeiro nome (String), Sobrenome (String) e salário mensal (double). Escreva um construtor que inicialize os três atributos e métodos set e get para cada um deles. Apenas valores positivos devem ser aceitos para o salário mensal. Escreva um aplicativo de teste nomeado TesteEmpregado que demonstre o comportamento da classe Empregado. Este aplicativo deve criar dois objetos empregado e exibir o salário anual dos mesmos. Na sequência, aumente em 10% o salário de cada empregado e exiba seus salários anuais novamente.

Programação Orientada a Objetos

Lista de Exercícios 02

Aula 04 – Campos e Métodos Estáticos

- 1) Implemente um método estático que use os métodos da classe `Math` para encontrar o maior e menor valor de um array de 5 elementos passado como parâmetro. Escreva um aplicativo para testar este método.
- 2) Implemente um método estático chamado *isMultiple* que determina para um par de números inteiros se o segundo número é múltiplo do primeiro. O método deve aceitar dois argumentos inteiros e retornar **true** se o segundo for um múltiplo do primeiro e **false** caso contrário. Incorpore este método a um aplicativo que leia uma série de pares inteiros (um par por vez) e determine se o segundo valor em cada par é um múltiplo do primeiro.
- 3) Um problema comum de uma mesa de bar é que todo mundo divide o que está sendo pedido mas ninguém sabe o valor que precisa pagar da conta na hora que vai embora: sempre tem um que chega mais tarde ou outro que sai mais cedo que o grupo, de forma que na hora de pagar há sempre a velha luta dos clientes com a conta no guardanapo de papel. Afinal, cada um quer pagar apenas sua parte do que foi consumido, não é? (Claro, adicionando-se os 10% do garçom...).

Uma maneira de resolver esse problema é que cada pessoa mantenha a informação de quanto lhe cabe pagar de cada pedido feito enquanto está na mesa. Assim, por exemplo, caso um cliente A tenha sentado numa mesa de amigos com seis pessoas e uma bebida que custa R7,00 tenha sido pedida (para ser dividida) logo em seguida, aquele cliente deve manter anotado que ele já tem em consumo R\$1,00 ($R\$7,00 / 7$ pessoas)). Assim como o cliente A, os outros clientes estão mantendo as suas contas atualizadas. Quando um cliente quer ir embora, este deve pagar sua parte (incluindo os 10%), sem maiores dificuldades.

Crie um aplicativo Java que simule esse comportamento. O aplicativo deve possibilitar ao usuário as opções “adicionar um cliente” (cliente senta na mesa), “remover um cliente” (cliente paga sua conta e vai embora) e “adicionar despesa” (alguém da mesa fez um pedido), além de uma opção para sair. Ao adicionar um cliente, deve ser dado um nome a ele para facilmente identificá-lo. Ao adicionar uma despesa, o usuário deve indicar o valor total desta despesa, a qual será atribuída proporcionalmente para cada cliente. Após cada ação do usuário, deve ser apresentado o valor acumulado de cada cliente presente na mesa.

Para este exercício, complete e implemente as classes `Cliente` e `Mesa` (Arquivo `ListaII-Exercicio3.rar`). Nota: a quantidade de clientes na mesa deve ser guardada como um atributo estático da classe `Cliente`!

Aula 05 – Arrays em Java

- 4) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
- Um vetor pode armazenar diferentes tipos de variáveis
 - O índice de acesso dos vetores deve ser do tipo float
 - Se um vetor for passado como parâmetro a um método com tipo de retorno void e um de seus elementos for modificado, ao final do método o vetor terá seu valor modificado.
 - Argumentos passados pela linha de comando são separados por vírgulas
 - Para reservar espaço na memória, basta eu declarar a variável.
- 5) Escreva um aplicativo que que leia uma determinada quantidade de números, entre 10 e 100. A cada número lido, o aplicativo deve exibir uma mensagem caso o número entrado seja um número repetido. Ao final, o aplicativo deve exibir os números que foram lidos somente uma vez.
- 6) Implemente um aplicativo para controlar os candidatos de uma prova de concurso público. O aplicativo deve armazenar os nomes dos candidatos inscritos no concurso. O número máximo de candidatos armazenados deve ser informado como argumento de linha de comando. O sistema deve ter as seguintes opções:
- 1 – Cadastrar candidato
 - 2 – Listar candidatos
 - 3 – Contar candidatos
 - 4 – Sair

A opção 1 (cadastrar candidatos) só deve permitir que um candidato seja cadastrado por vez. Para cadastrar mais de um candidato, essa opção precisa ser usada múltiplas vezes.

- 7) Implemente um aplicativo que a leitura de duas matrizes A e B e posteriormente informe o resultado da soma das mesmas. A quantidade de linhas e colunas das matrizes deve ser passada pelo usuário como um argumento da linha de comando, e o valor de cada elemento deve ser lido do teclado.

Programação Orientada a Objetos**Lista de Exercícios 03****Aula 06 – Herança**

- 1) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
 - a) Construtores das superclasses não são herdadas por subclasses
 - b) Um relacionamento parte-de é implementado via herança
 - c) Uma classe Carro tem um relacionamento é-um com as classes Volante e Freios
 - d) Quando uma subclasse redefine um método da superclasse usando a mesma assinatura, ela é dita por sobrecarregar o método da superclasse.
- 2) Discuta de que maneira a herança promove a reutilização de software, economiza tempo durante o desenvolvimento de um programa e ajuda a evitar erros.
- 3) Discuta como é feita a chamada de construtores em subclasses. O código abaixo irá funcionar? Justifique.

```
public class TesteConstrutorPaiFalho{
    public TesteConstrutorPaiFalho(String nomePai){
        System.out.println("Construtor pai falho: "+ nomePai);
    }
}

public class TesteConstrutorFilhoFalho extends TesteConstrutorPaiFalho {
    public TesteConstrutorFilhoFalho (){
        System.out.println("Construtor filho falho");
    }

    public static void main(String args[]){
        TesteConstrutorFilhoFalho t = new TesteConstrutorFilhoFalho();
    }
}
```

- 4) Escreva uma hierarquia de herança para as classes Quadrilateral, Trapezoid, Parallelogram, Rectangle e Square. Use Quadrilateral como a superclasse da hierarquia. Crie e use uma classe Point para representar os pontos em cada forma. Faça a hierarquia o mais profunda possível, isto é, com muito níveis. Especifique as variáveis de instância e os métodos para cada classe. As variáveis de instância private de Quadrilateral devem ser os pares de coordenadas x-y para os quatro pontos que delimitam o Quadrilateral. Escreva um programa que instancia objetos de suas classes e gera saída da área de cada objeto (exceto Quadrilateral).



- 5) Elabore um aplicativo de simulação de controle de contas bancárias. Todas as contas devem ter o nome do cliente, número da conta, saldo e métodos para sacar e depositar o dinheiro. Além disso, especialize a classe para que possa representar diferentes tipos de contas, neste caso, conta poupança e conta especial. Conta poupança possui um dia de rendimento e método de calcular novo saldo, dado o rendimento (%). Saques não podem deixar poupança negativa. Cada conta especial possui um limite que pode ser sacado depois do saldo estar zerado. Crie um aplicativo para testar a classe com um menu que receba o nome do usuário. Caso usuário já exista, abra outro menu que o permita sacar, depositar ou verificar dia de rendimento (caso conta seja do tipo poupança). Caso usuário não exista, abra opção de criar um dos dois tipos de conta. Caso o usuário entrada seja "admin", liste todos os correntistas com seus respectivos tipos de conta.

Aula 07 – Polimorfismo, Classes Abstratas, Interfaces e Exceções

- 6) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
- Todos os métodos em uma classe abstrata devem ser declarados como métodos abstratos
 - Invocar um método exclusivo da subclasse por uma variável do tipo da subclasse não é permitido
 - Se uma superclasse declara um método abstrato, uma subclasse deve implementar aquele método
 - Um objeto de uma classe que implementa uma interface pode ser considerado como um objeto daquele tipo da interface.
- 7) Quais tipos de exceção podem ser capturadas utilizando o bloco abaixo? Por que não é adequado utilizar este tipo de tratamento de exceções?

```
try{
    } catch(Exception e){}
```

- 8) Uma companhia paga seus empregados em uma base semanal. Os empregados são de quatro tipos. Empregados salarizados são pagos um salário semanal independentemente do horário de horas trabalhadas. Empregados por hora são pagos pela quantidade de horas trabalhadas e recebem hora extra (1,5x o salário de uma hora tradicional) para cada hora depois de 40 horas trabalhadas. Empregados de comissão são pagos por uma porcentagem de suas vendas e um empregado de comissão salarizado a porcentagem de suas vendas mais um salário base. Escreva um método que receba qualquer um destes objetos e calcule seu pagamento semanal. (Não sobrecarregue os métodos! Implemente somente um método que trate esses objetos de forma polimórfica.)
- 9) Altere o exercício 8 (ganhos semanais para os empregados) e crie uma classe de exceção `IllegalWorkingConditions`. No método de calcular os ganhos de cada tipo de empregado, avalie se os empregados de comissão recebem menos que 10% de suas vendas e se os empregados por hora trabalharam mais de 60 horas na semana. Caso alguma dessas situações sejam encontradas, lance a exceção que você acabou de criar e a trate adequadamente. Insira uma `String` na exceção, explicando a exceção que ocorreu e a exiba para o usuário no bloco `catch`. Para avisar que os ganhos de algum empregado já foram calculados, crie um bloco `finally` junto ao `try-catch` que, ao terminar o cálculo de ganhos, seja normalmente ou depois de avisar ao usuário de que o empregado está em situação de trabalho irregular, avise que os seus ganhos já estão calculados.

Nota: Ao se criar um aplicativo Java, é importante não permitir que o programa pare de executar, a não ser que o próprio usuário solicite isto (o mesmo deverá ser válido para o projeto). Portanto, neste exercício, caso alguma exceção seja lançada para algum empregado, avise o usuário com uma mensagem, não calcule os ganhos deste empregado, mas continue calculando dos outros em situações regulares de trabalho.

Programação Orientada a Objetos**Lista de Exercícios 04****Aula 08 – Arquivos e Fluxos**

- 1) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
 - a) O método `exists()` da classe `File` retorna verdadeiro se o nome especificado como argumento para o construtor do `File` é um arquivo ou diretório no caminho especificado.
 - b) Arquivos binários são legíveis por humanos em um editor de texto.
 - c) Ao se criar um objeto do tipo `File`, você automaticamente cria um arquivo no computador.

- 2) Em diversos projetos de análise de dados é dado uma grande importância para o pré-processamento dos dados, para se ter uma ideia inicial do que está presente nos dados. Em <http://archive.ics.uci.edu/ml/machine-learning-databases/statlog/heart/heart.dat> estão presentes dados obtidos de pacientes que tiveram doenças cardíológicas. Foram coletados 13 informações de um total de 270 pacientes e, mais uma informação dizendo se o dado paciente teve ou não uma doença cardíológica (1 = ausência 2 = presença de doença cardíaca). As informações da base de dados e o significado de cada dado coletado dos pacientes podem ser vistos em <http://archive.ics.uci.edu/dataset/145/statlog+heart>. Escreva uma aplicação Java para realizar uma pré-análise dos dados. Essa aplicação deve ler o arquivo fornecido e indicar:
 - a) Quantidade de pacientes que possuem uma doença cardíaca;
 - b) Quantidade de pacientes que não possuem uma doença cardíaca;
 - c) Quantidade de pessoas nas seguintes faixas etárias:
 - i) 40-50 anos
 - ii) 50-60 anos
 - iii) 60-70 anos
 - d) Quantidade de pessoas com dores no peito nível 1 e 2 e que tiveram alguma doença cardíaca;
 - e) Quantidade de pessoa com dores no peito nível 3 e 4 e que tiveram alguma doença cardíaca;

Armazene em um arquivo texto chamado “`resultadoPreAnaliseDoencasCardiacas.txt`” os resultados obtidos da sua análise no exercício anterior. Para facilitar a leitura por qualquer pessoa interessada, a estrutura do arquivo deve conter a pergunta que foi realizada no exercício anterior, seguida da resposta. Por exemplo:

“a) Quantidade de pacientes que possuem uma doença cardíaca: 400”

- 3) Considere o exercício 5 da Lista III. Acrescente uma opção “Salvar e sair do sistema”. Nesta opção, todos os clientes com contas devem ser serializados em arquivos separados (com a extensão `.serialized`) dentro de um diretório chamado `clientData`. Quando o sistema (re)iniciar, este deve recuperar todo o conteúdo armazenado no diretório `clientData`.

Aula 09 – Métodos e Classes Genéricas

- 4) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
- a) Um método genérico não pode ter o mesmo nome de método de um método convencional, isto é, não genérico.
 - b) Todas as declarações de métodos genéricos têm uma seção de parâmetro de tipo que precede imediatamente o nome de método.
 - c) Um método genérico pode ser sobrecarregado por outro método genérico com o mesmo nome de método, mas diferentes parâmetros de método.
 - d) Um parâmetro de tipo pode ser declarado somente uma vez na seção de parâmetro de tipo, mas pode aparecer mais de uma vez na lista de parâmetros do método.
 - e) Os nomes dos parâmetros de tipo entre diferentes métodos genéricos devem ser únicos.
 - f) O escopo de um parâmetro de tipo da classe genérico é a classe inteira, exceto seus membros static.
- 5) Crie uma classe genérica Dictionary que representa um mapeamento entre dois elementos. Esta classe pode ser utilizada para conter diversas instâncias da classe Pair (desenvolvida no exercício dado em sala de aula). Dictionary deve ser inicializada com os parâmetros de tipo F e S também, de forma que todos os pares que o Dictionary contém tenham os mesmos tipos para o primeiro e para o segundo elemento. Dictionary deve ter um método add(F, S) que pode ser utilizado para adicionar um Pair à instância. Dictionary também deve ter um método para retornar um Pair a partir de uma dada chave (primeiro elemento do par).

Nota: Não deve haver duas instâncias de Pair com mesma chave (primeiro elemento do par) em um único dicionário. Assim, a cada adição de uma nova instância de Pair a uma instância de Dictionary, essa instância de Dictionary deverá comparar a chave do novo par à chave de todos os pares que guarda.



Aula 10 – Coleções

- 6) As seguintes afirmações são verdadeiras ou falsas? Justifique as alternativas falsas.
- a) Valores primitivos podem ser armazenados diretamente em coleções.
 - b) Collections é uma interface.
 - c) Para percorrer coleções, são usados objetos da classe Iterator.
- 7) Considere o exercício 3. Implemente um método estático que retorne uma lista contendo as idades de todos participantes no estudo. Além disso, apresente os resultados para o usuário de forma ordenada, crescente.

Nota: Utilize a interface Set para não inserir elementos repetidos e Collections para ordenação.

- 8) Escreva um aplicativo que leia um conjunto de nomes (primeiro nome de uma pessoa, por exemplo) e músicas favoritas. Utilize um HashMap para armazenar um nome e relacioná-lo à música preferida. A aplicação deve permitir a adição de novos nomes (e respectiva música preferida), a verificação da música preferida de uma pessoa dado seu nome e a modificação da música preferida de uma pessoa.